Programmazione Concorrente

Concetti fondamentali e terminologia

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

27 aprile 2026

UniMe
1548

Un sistema che fa più cose insieme

La **programmazione concorrente** è la disciplina che studia come strutturare un programma in modo che più attività possano *avanzare contemporaneamente* nel tempo, indipendentemente dal numero di processori disponibili.

Non si tratta soltanto di velocità: la concorrenza è prima di tutto un **modo di ragionare e strutturare** il software. Un programma concorrente decompone il problema in *task* che possono progredire autonomamente, comunicando e sincronizzandosi quando necessario.

Definizione operativa

Un programma è **concorrente** se contiene più flussi di esecuzione logicamente indipendenti che cooperano verso un obiettivo comune.

L'etimologia del termine

La parola **concorrente** viene dal latino *concurrere*: **con-** (insieme) + **currere** (correre). L'immagine è quella di più entità che *corrono insieme* verso il loro completamento.

In italiano il termine porta con sé una seconda sfumatura semantica: la **competizione**. E in effetti anche questo senso è pertinente, perché i task *competono* per le risorse condivise — CPU, memoria, lock, connessioni.

Nota linguistica

L'inglese *concurrent* conserva lo stesso significato: "che scorre insieme". In diritto penale anglosassone, *concurrent sentences* indica pene che si scontano simultaneamente.

Il termine cattura quindi due verità: i task **avanzano insieme**, ma allo stesso tempo **si contendono** le risorse. Entrambe le sfumature descrivono qualcosa di reale nel modello concorrente.

Tre ragioni fondamentali

Velocità

Distribuire il calcolo su più core, se disponibili, riduce il tempo di esecuzione per task **CPU-bound**: compressione, crittografia, rendering, machine learning.

Efficienza

Non sprecare tempo ad aspettare I/O. Mentre un thread attende la rete o il disco, un altro usa la CPU. Il tempo di attesa viene **sovrapposto**.

Reattività

Separare il thread UI dal thread di lavoro mantiene l'interfaccia **sempre reattiva**, anche durante operazioni lunghe.

Il quarto motivo: modellare la realtà

Il mondo reale è intrinsecamente concorrente. In un ospedale più medici visitano più pazienti simultaneamente; in una città milioni di eventi accadono in parallelo. Certi sistemi — simulazioni, videogiochi, microservizi — si modellano molto più naturalmente con la concorrenza.

Due concetti distinti

La citazione fondamentale

*"Concurrency is about **dealing with** lots of things at once. Parallelism is about **doing** lots of things at once."* — Rob Pike, 2012

Concorrenza è una questione di *struttura*. Un sistema concorrente è progettato per gestire più task sovrapposti nel tempo, ma in ogni istante fisico può eseguirne uno solo (su single core). È un modo di **organizzare** il programma.

Parallelismo è una questione di *esecuzione*. Richiede hardware dedicato — più core, più CPU — e permette a più task di eseguire **nello stesso istante fisico**. È una proprietà della macchina, non del programma.

$$\text{Concorrenza} \supset \text{Parallelismo}$$

L'analogia del ristorante

Un cameriere

Concorrente, non parallelo.

Il cameriere non serve dieci clienti nello stesso istante, ma avanza su tutti i tavoli in modo *interleaved*: prende l'ordine al tavolo 1, porta il conto al tavolo 3, serve il vino al tavolo 7.

Un solo esecutore, molti task sovrapposti.

Dieci camerieri

Parallelo (e concorrente).

Ogni cameriere lavora sul proprio tavolo *nello stesso momento*. Qui la simultaneità è fisica, non simulata.

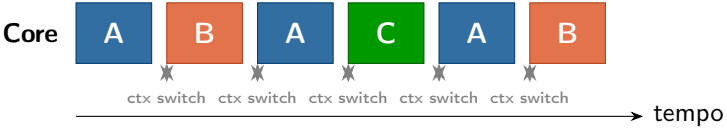
La concorrenza ha dato la struttura; il parallelismo fornisce i "processori".

La lezione pratica

Non si può parallelizzare qualcosa che non è stato prima strutturato in modo concorrente. Ma si può avere concorrenza senza parallelismo — ed è spesso tutto ciò di cui si ha bisogno per sistemi I/O-bound.

Single core: la concorrenza simulata

Su un singolo core **un solo thread è in esecuzione in ogni istante fisico**. La concorrenza è un'illusione costruita dallo scheduler tramite il *context switch*: lo stato completo del thread (registri, program counter, stack) viene salvato e ripristinato decine o centinaia di volte al secondo.

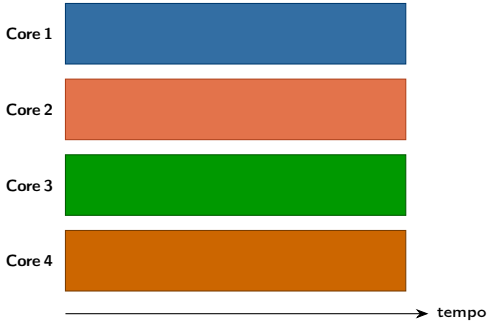


Attenzione

Le race condition su single core sono più rare ma **non impossibili**: un context switch nel momento sbagliato le scatena. Codice che "funziona" su single core può rompersi silenziosamente su multi core.

Multi core: la concorrenza reale

Su più core il parallelismo è fisico. Due thread eseguono istruzioni **nello stesso istante**, su core diversi. Questo cambia tutto.



Ogni core può eseguire un thread diverso in parallelo. Le race condition diventano **molto più frequenti** perché l'interleaving non è più controllato dallo scheduler ma avviene fisicamente.

Inoltre ogni core ha la propria **cache L1/L2**: una scrittura del Core 1 potrebbe non essere immediatamente visibile al Core 2 — serve sincronizzazione della memoria.

Legge di Amdahl

La legge di Amdahl

Con una frazione parallelizzabile P e N core, lo speedup massimo è
$S = \frac{1}{(1-P) + P/N}$.

La legge di Amdahl

La legge di Amdahl quantifica il guadagno di velocità (*speedup*) ottenibile parallelizzando un programma su N core, quando una frazione $(1 - P)$ del codice è **intrinsecamente sequenziale** e non può essere parallelizzata.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Immaginiamo un programma che gira in **100 secondi** su un core:

- **20s** — lettura, parsing, scrittura: sequenziale
- **80s** — elaborazione dati: parallelizzabile

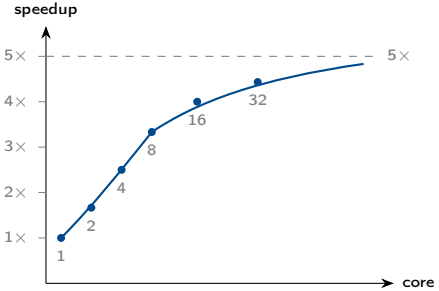
Quindi $P = 0.80$ e $(1 - P) = 0.20$.

Il limite invalicabile

Con infiniti core la parte parallelizzabile tende a zero, ma la parte sequenziale rimane costante. Speedup massimo teorico: $\frac{100}{20} = 5\times$

La legge di Amdahl

Core N	Parte par.	Totale	Speedup
1	80s	100s	1.00×
2	40s	60s	1.67×
4	20s	40s	2.50×
8	10s	30s	3.33×
16	5s	25s	4.00×
32	2.5s	22.5s	4.44×
∞	$\approx$ 0s	20s	**5.00**×



La legge di Amdahl

Il rendimento marginale decresce

Passare da 1 a 2 core guadagna **40 secondi**.
Passare da 16 a 32 core guadagna **2.5 secondi**.

Ogni core aggiuntivo contribuisce sempre meno. Oltre una certa soglia, aggiungere hardware non produce benefici apprezzabili.

Ottimizzare la parte sequenziale vale di più

Ridurre $(1 - P)$ dal 20% al 5% sposta il limite asintotico da $5\times$ a $20\times$ — un guadagno che nessuna quantità di core può ottenere.

La lezione architettuale

La parte sequenziale — lettura da disco, parsing, sincronizzazione, I/O — è il **collo di bottiglia assoluto**. Prima di aggiungere core, bisogna chiedersi:

- Quanto è grande $(1 - P)$?
- Posso ridurlo?
- Il mio problema è davvero CPU-bound?

Atomicità e visibilità

Atomicità

Un'operazione è **atomica** quando è indivisibile: o avviene completamente, o non avviene affatto. L'innocuo `x = x + 1`

si traduce in tre istruzioni macchina (LOAD, ADD, STORE). Due thread che lo eseguono sulla stessa area di memoria possono confliggere.

Visibilità della memoria

In un sistema multi-core, ogni core ha la propria cache. Una scrittura di un thread **non è immediatamente visibile** agli altri: la parola scritta può restare nella cache locale per un tempo indefinito.

I linguaggi moderni risolvono questo con *memory barriers*, variabili `volatile` (Java) e tipi `atomic` (C++/Rust).

Race condition, Deadlock, Liveness

Race condition

Si ha una **race condition** quando la correttezza del risultato dipende dall'ordine temporale con cui i thread accedono a dati condivisi — ordine che il programma non controlla né garantisce. I bug risultanti sono **non deterministici**: stessi input, risultati diversi a ogni esecuzione.

Deadlock

Sia ha un **Deadlock** quando due o più thread si bloccano aspettando ognuno una risorsa tenuta dall'altro.

Liveness e sue violazioni

Liveness (Vivezza)

Ogni task che vuole progredire prima o poi lo fa, in qualsiasi esecuzione sufficientemente lunga del sistema. Le violazioni includono la *starvation* e il *livelock*.

Starvation

Il progresso non avviene per un thread specifico, anche se il sistema nel suo complesso avanza. Il thread in question è affamato di risorse.

Livelock

Il progresso non avviene nonostante i thread si muovano. È la forma più sottile: i thread eseguono istruzioni, cambiano stato, consumano CPU — ma girano in cerchio senza avvicinarsi mai al loro obiettivo. È una violazione ingannevole: il sistema sembra vivo ma è semanticamente bloccato.

Memoria condivisa

Minimizzare lo stato condiviso mutabile

La radice di quasi tutti i problemi della concorrenza è la **memoria condivisa**. Ogni dato condiviso è un potenziale punto di contesa.

Le strategie per ridurre il problema alla radice:

- **Immutabilità**: Un dato immutabile non può essere modificato dopo la sua creazione — quindi può essere condiviso liberamente tra quanti thread si vuole, senza alcuna sincronizzazione.
- **Thread-local**: Invece di condividere un dato tra tutti i thread, ogni thread ha la propria copia privata. Non c'è condivisione, quindi non c'è contesa — per definizione (thread_local in C11)
- **Message Passing**: Invece di condividere memoria, i thread non si vedono affatto — comunicano scambiandosi messaggi attraverso canali. Il dato non viene condiviso: viene trasferito da un thread all'altro. In ogni momento, un dato appartiene a un solo thread.
- **Compile-time ownership**: Rust porta il concetto di ownership al livello del sistema dei tipi, garantendo a compile time che un dato abbia sempre esattamente un proprietario. Non è una convenzione o una disciplina di programmazione — è una regola che il compilatore verifica e fa rispettare.

Due unità di esecuzione

Processo

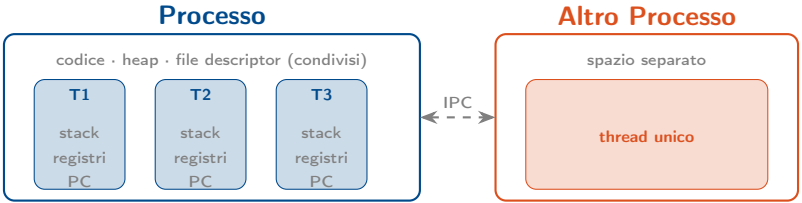
Un processo è un programma in esecuzione con il proprio **spazio di indirizzamento isolato**: codice, stack, heap, file descriptor. Il kernel gestisce l'isolamento.

La comunicazione tra processi (IPC) richiede meccanismi espliciti: pipe, socket, shared memory. Il fallimento di un processo non corrompere gli altri.

Thread

Un thread è un flusso di esecuzione **all'interno** di un processo. Tutti i thread dello stesso processo condividono codice, heap e file descriptor; ognuno ha il proprio stack e registri.

La comunicazione è immediata (memoria condivisa), ma un thread difettoso può corrompere l'intero processo.



Confronto pratico

	Thread	**Processo**
Memoria	Condivisa per default	Separata; IPC esplicita
Creazione	Rapida (pochi μs)	Lenta (`fork` copia lo spazio)
Context switch	Leggero	Pesante
Comunicazione	Diretta (puntatori condivisi)	IPC: pipe, socket, shm
Isolamento	Nessuno: un bug può corrompere tutto	Forte: crash isolato
Caso d'uso tipico	Server ad alto throughput, calcolo parallelo	Componenti isolati, microservizi
API Linux	`pthread_create`, `clone`	`fork`, `exec`

Dal hardware al kernel: i livelli

IPC / I/O asincrono pipe · socket · shm · eventfd · epoll · io_uring

↑

POSIX pthreads mutex · spinlock · rwlock · cond. variable · barrier · semafori

↑

Futex fast userspace mutex — base di tutto il resto

↑

Hardware operazioni atomiche (CAS, fetch-add)

astrazione crescente ↑

Strumenti 1: Mutex, Spinlock e Operazioni Atomiche

Mutex `pthread_mutex_t`

Garantisce che un solo thread alla volta esegua la sezione critica. Il thread in attesa **dorme** (cedendo la CPU).

Spinlock `pthread_spinlock_t`

Come il mutex, ma il thread attende in un **loop attivo** (busy-wait). Zero overhead di sospensione, ma spreca cicli CPU.

Atomiche `<stdatomic.h>`

Operazioni indivisibili garantite dall'hardware. Nessun lock, nessun context switch. `atomic_fetch_add`, `atomic_compare_exchange` (CAS). Fondamento degli algoritmi lock-free. Per singole variabili: contatori, flag, puntatori.

Il Futex sotto il cofano

Mutex e spinlock di pthreads sono costruiti sui **futex** (fast userspace mutex). Nel caso non conteso tutto avviene in userspace con un'operazione atomica — zero syscall. La syscall kernel viene invocata solo quando il thread deve davvero dormire.

Futex — Fast Userspace Mutex

Un futex è un **intero in memoria condivisa** tra i thread, letto e modificato con operazioni atomiche in userspace — senza mai coinvolgere il kernel nel caso comune.

Il kernel viene chiamato **solo quando un thread deve davvero dormire**: quando il lock è conteso, una syscall mette il thread in una coda di attesa associata all'indirizzo di quell'intero. Un'altra syscall lo sveglia quando il lock viene rilasciato.

Caso non conteso → userspace, zero syscall, ∼5 nanosecondi.

Caso conteso → kernel, syscall, thread dorme.

Sono il mattone su cui Linux costruisce tutto il resto: mutex, condition variable, semafori, read-write lock.

Strumenti 2: Read-Write Lock, Condition Variable, Barrier

RW Lock
`pthread_rwlock_t`

Ottimizzazione per scenari con **molte letture e poche scritture**. Più lettori possono accedere contemporaneamente; uno scrittore ha accesso esclusivo.

Attenzione: i writer possono soffrire di *starvation* se i lettori sono continui.

Condition Variable
`pthread_cond_t`

Notifica che una condizione è cambiata. Il thread dorme finché non viene svegliato da `signal` o `broadcast`.

Va **sempre** usata con un mutex e un `while` (non `if`): i *spurious wakeup* sono reali.

Barrier `pthread_barrier_t`

Punto di sincronizzazione collettivo: tutti i thread devono arrivarci prima che qualsiasi thread possa proseguire.

Fondamentale negli algoritmi paralleli a fasi: map-reduce, simulazioni fisiche, elaborazione di immagini.

Semafori: le primitive di Dijkstra

I semafori furono introdotti da **Edsger W. Dijkstra** nel 1965 nel sistema operativo THE. Sono il primo meccanismo di sincronizzazione formalmente definito.

Le due operazioni originali, in olandese:

Nome	Olandese	Effetto
`P(S)`	*Proberen*	$S \leftarrow S - 1$; blocca se $S = 0$
`V(S)`	*Verhogen*	$S \leftarrow S + 1$; sveglia un thread

Entrambe le operazioni sono **atomiche**. A differenza del mutex, il semaforo non ha il concetto di *proprietario*: chiunque può fare `V`.

Semaforo POSIX su Linux

`sem_init(&s, 0, N)` — valore iniziale N
`sem_wait(&s)` — corrisponde a `P`
`sem_post(&s)` — corrisponde a `V`

I **semafori named** (`sem_open`) permettono la sincronizzazione tra **processi diversi**, identificati da un path nel filesystem.

Semaforo binario ($N = 1$): si comporta come un mutex.
Semaforo contatore ($N > 1$): limita l'accesso concorrente a N risorse (es. pool di connessioni al database).

I semafori sono pericolosi!!

I semafori di Dijkstra sono potenti ma pericolosi — la correttezza del programma dipende **interamente dalla disciplina del programmatore**. Ogni P e ogni V deve essere nel posto giusto, nell'ordine giusto. Un errore è silenzioso
Il compilatore non dice nulla. Il bug emerge in modo non deterministico a runtime.
Dijkstra stesso era consapevole del problema — i semafori sono troppo vicini al livello macchina per essere usati in modo affidabile su larga scala.
Hoare nel 1974 si pone una domanda precisa: è possibile strutturare il codice in modo che la mutua esclusione sia garantita dalla struttura stessa, senza che il programmatore debba ricordarsene?
La risposta è il monitor.

I Monitor di Hoare

Tony Hoare (1974) propose i *monitor* come alternativa più sicura ai semafori: la mutua esclusione è **garantita dalla struttura**, non dal programmatore.

Un monitor incapsula dati condivisi e operazioni, garantendo che *un solo thread alla volta* sia attivo al suo interno. Le **condition variable** di Hoare permettono l'attesa selettiva:

`wait(c)`	rilascia il monitor, dorme su `c`
`signal(c)`	sveglia un thread in attesa su `c`

Hoare propose che `signal` cedesse immediatamente il controllo al thread svegliato (*semantica Hoare*). I sistemi moderni usano la **semantica Mesa**: il thread svegliato viene messo in coda, il signaler continua.

I monitor oggi

I monitor di Hoare sono vivi in ogni linguaggio moderno:

- **Java**: `synchronized` + `wait()` / `notify()`
- **C#**: `lock` + `Monitor.Wait()`
- **Python**: `threading.Condition`
- **C (pthreads)**: `pthread_mutex` + `pthread_cond`

I monitor

Un monitor è una struttura che incapsula insieme dati condivisi e le operazioni su di essi, con una garanzia fondamentale incorporata: un solo thread alla volta può essere attivo all'interno del monitor. Ha tre componenti:

- **Dati privati** — le variabili condivise sono nascoste all'interno del monitor. Nessun thread può accedervi direttamente dall'esterno.
- **Procedure** — le uniche operazioni sui dati condivisi sono le procedure del monitor. Sono l'unica interfaccia verso l'esterno.
- **Mutua esclusione automatica** — quando un thread entra in una procedura del monitor, acquisisce implicitamente un lock. Quando esce, lo rilascia. Non serve chiamare esplicitamente nulla.

Il problema dell'uscita dal monitor

Un monitor garantisce la mutua esclusione automatica — un solo thread alla volta è attivo al suo interno. Ma questo da solo non basta. Immaginiamo un consumatore che entra nel monitor per prelevare un dato da un buffer. Il buffer è vuoto. Cosa fa? Non può prelevare — la condizione non è soddisfatta. Non può restare dentro il monitor ad aspettare — terrebbe il lock e nessun produttore potrebbe mai entrare per inserire dati. Deadlock garantito. Il monitor ha bisogno di un meccanismo che permetta a un thread di uscire temporaneamente dall'esclusione mutua, aspettare che una condizione cambi, e poi rientrare. Questo meccanismo sono le condition variable con le operazioni wait() e signal().

IPC e I/O asincrono: pipe, socket, epoll, io_uring

Comunicazione tra processi

Pipe / FIFO: canale unidirezionale. La pipe anonima funziona tra processi correlati; la FIFO (named pipe) tra processi qualsiasi tramite un path nel filesystem.

Unix domain socket: IPC bidirezionale sulla stessa macchina, prestazioni vicine alla memoria condivisa.

Shared memory (`shm_open` + `mmap`): il segmento più veloce possibile, ma va sempre protetto con semafori o mutex.

I/O asincrono ad alte prestazioni

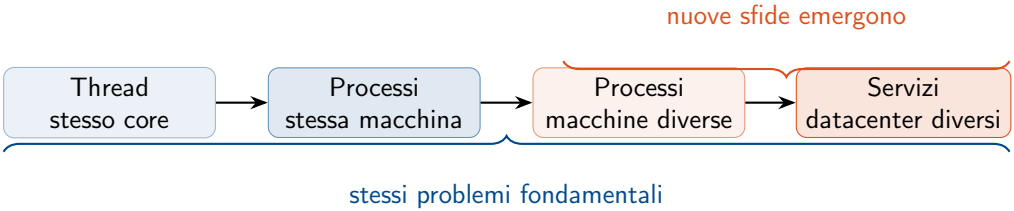
epoll: monitora migliaia di file descriptor senza bloccarsi. Complessità $O(1)$ sugli eventi attivi — base di Node.js, Nginx, quasi ogni server ad alte prestazioni su Linux.

io_uring (kernel 5.1, 2019): I/O asincrono con zero syscall nel caso ottimale, tramite due ring buffer condivisi tra userspace e kernel. Rappresenta il futuro dell'I/O su Linux.

eventfd: contatore condiviso tra thread o processi, integrabile con epoll per notifiche efficienti.

La concorrenza scala fino ai sistemi distribuiti

La programmazione concorrente e i sistemi distribuiti non sono discipline separate: i sistemi distribuiti sono la **concorrenza portata all'estremo**, con sfide nuove che non esistono su una singola macchina.



I problemi classici riappaiono travestiti da problemi di rete: race condition → conflitti di scrittura distribuita; deadlock → deadlock distribuito tra microservizi; sincronizzazione → protocolli di consenso.

I problemi nuovi: fallimenti parziali e assenza di clock globale

Fallimenti parziali

Su una singola macchina il fallimento è binario: o tutto funziona o il processo crasha. In un sistema distribuito si può avere: il nodo A vivo, il nodo B morto, il nodo C vivo ma lentissimo, la rete tra A e C con packet loss del 10%.

La domanda impossibile su singola macchina: **"Il nodo ha crashato o è semplicemente lento?"**

Assenza di clock globale

Su una singola macchina tutti i thread condividono lo stesso orologio. Su macchine diverse i clock **divergono** nel tempo — anche con NTP la sincronizzazione è approssimata.

Soluzione: i **Logical Clock** di Leslie Lamport (1978). Invece del tempo fisico si usano contatori logici che catturano le relazioni causali tra eventi con la relazione *happens-before* ($\rightarrow$).

Il teorema CAP (Brewer, 2000)

Un sistema distribuito non può garantire simultaneamente: **Consistency** (ogni lettura vede l'ultima scrittura), **Availability** (ogni richiesta riceve risposta), **Partition Tolerance** (funziona anche con nodi disconnessi). Deve scegliere due su tre.

Tre giganti: Dijkstra, Hoare, Lamport

Dijkstra

Semafori P/V (1965)

Problema dei filosofi

Prima teoria formale

della concorrenza

"Come sincronizzare"
su singola macchina

Hoare

Logica {P} C {Q} (1969)

Monitor (1974)

Condition variable

Correttezza formale

"Come strutturare"
e ragionare

Lamport

Logical Clock (1978)

Byzantine Generals (1982)

Paxos (1989)

TLA+ (1999)

"Come coordinare"
su macchine diverse

Fondamenta teoriche della programmazione concorrente e distribuita

Riferimenti bibliografici

- E. W. Dijkstra, *Solution of a problem in concurrent programming control*, Communications of the ACM, 8(9), 1965.
- C. A. R. Hoare, *An Axiomatic Basis for Computer Programming*, Communications of the ACM, 12(10), 1969.
- C. A. R. Hoare, *Monitors: An Operating System Structuring Concept*, Communications of the ACM, 17(10), 1974.
- L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, Communications of the ACM, 21(7), 1978.
- L. Lamport, R. Shostak, M. Pease, *The Byzantine Generals Problem*, ACM TOPLAS, 4(3), 1982.
- L. Lamport, *The Part-Time Parliament*, ACM TOCS, 16(2), 1998.
- R. Pike, *Concurrency is not Parallelism*, talk at Heroku's Waza conference, 2012.